SENTINEL AI

AI Data Leakage Prevention Gateway — 3-Member Engineering Blueprint & AI Prompts

Implementation Plan, Modular Responsibility Division, Separation of Concerns, and Turnkey AI Coding Prompts for pair programming with Cursor, ChatGPT-4o, Claude 3.5 Sonnet, and Antigravity.

Specification: PRD/SRS v2.0 MVP **Team Size:** 3 Members **Date:** 15 October 2026 **Status:** Ready for Sprint 1

1 Executive Architecture & Core Security Guarantees

Sentinel AI is an enterprise application-layer security gateway mediating all interactions between authenticated client applications and Large Language Model (LLM) providers. It systematically eliminates the three primary channels of AI data leakage:

- **Input Leakage:** Sensitive PII (emails, phone numbers, Indian Aadhaar & PAN cards) and credentials (AWS keys, API tokens) pasted into prompts.
- **Retrieval Leakage:** Unauthorized document snippets fetched during RAG and injected into model context.
- **Output Leakage:** Model repeating internal credentials or memorized sensitive data in generated answers.

```
User Prompt (Client) │ ▼ [1. Authentication] ──➤ Verify user identity & derive tenant / doc scopes │ ▼ [2. Input Inspection] ──➤ Presidio + Regex: Email, Phone, Indian Aadhaar/PAN, AWS/API Keys, Injections │ ▼ [3. Policy Engine] ──➤ Evaluate ALLOW / MASK / BLOCK │ │ │ └➤ IF BLOCK ──────➤ 🔴 TERMINATE IMMEDIATELY (Upstream Provider is NEVER called) │ ▼ [4. Pre-Retrieval ACL] ➤ Verify document permissions server-side BEFORE retrieval │ ▼ [5. LLM Adapter] ──────➤ Call OpenAI / Groq / Anthropic or Offline Mock with Server-side Key │ ▼ [6. Output Filter] ──➤ Inspect generated response; redact PII or block credential leaks │ ▼ [7. Privacy Audit] ──➤ Store structured metadata (ZERO raw prompt or response in DB) │ Sanitized User Response
```

> **The Cardinal Rule: Block-Before-Forward (FR-005)**
> If a prompt receives a `BLOCK` decision due to credentials, API keys, or prompt injections, the upstream LLM provider **must never be invoked**. Automated security tests must strictly assert that provider call count is zero.

2 3-Member Work Division & Responsibility Matrix

To ensure zero merge conflicts and allow each team member to work independently using AI code generation tools, the codebase is partitioned into three decoupled modules:

Member	Operational Role	Owned Codebase Modules	Primary Deliverables

Member 1	SECURITY ARCHITECT	`backend/auth/`
		`backend/detection/`
		`backend/policy/`
		`backend/gateway/orchestrator.py`
			Input screeners (Aadhaar, PAN, Emails, AWS keys, Prompt Injections); Policy Engine (ALLOW/MASK/BLOCK); Masking transformer; **Block-Before-Forward** test.

Member 2	INTEGRATION ENGINEER	`backend/providers/`
		`backend/access_control/`
		`backend/retrieval/`
		`backend/output_filter/`
		`backend/api/routes.py`
			Server-side LLM Adapter (OpenAI/Groq/Mock); Pre-retrieval Document ACL; Post-generation Output Leak Filter; REST API chat and health endpoints.

Member 3	FULL-STACK & QA	`backend/audit/` `backend/evaluation/`	Privacy-Preserving Audit Logger (Zero-raw-leak); 120+ Case Evaluation Benchmark

Member	Operational Role	Owned Codebase Modules	Primary Deliverables
		`frontend/` (Web UI) `deploy/` & `compose.yaml`	(Precision/Recall); Interactive Chat & Inspector UI; Docker Compose.

3 Member 1 Deep Dive: Security Architect

Primary Objective: Intercept prompts, detect sensitive entities, apply policy precedence, and guarantee that blocked prompts never leave the server.

Owned Files & Structure:

- `backend/auth/models.py` & `backend/auth/jwt.py`: JWT decoding and role claims (`end_user`, `policy_admin`).
- `backend/detection/pii_detector.py`: Regex and Presidio detectors for Email, Phone, Indian Aadhaar (12 digits), Indian PAN (`[A-Z]{5}[0-9]{4}[A-Z]`), Credit Cards, and Hinglish markers.
- `backend/detection/secret_scanner.py`: Regex patterns for AWS keys (`AKIA...`), OpenAI keys (`sk-...`), GitHub tokens, passwords.
- `backend/detection/injection_detector.py`: Heuristic detector for prompt jailbreaks, overrides, and system prompt leaks.
- `backend/policy/engine.py`: Precedence evaluation and right-to-left deterministic masking transformer.
- `backend/gateway/orchestrator.py`: Central gateway orchestrator.
- `tests/unit/test_detection.py` & `tests/security/test_block_before_forward.py`: Unit and security tests.

Member 1 AI Prompts (Copy-Paste Ready):

PROMPT 1.1: PII & Indian Identifier Detector (Aadhaar, PAN, Phone) — **Target: Cursor / Claude 3.5 Sonnet**

```
You are an expert Python security engineer working on Sentinel AI (an AI Data Leakage Prevention Gateway).
Write `backend/detection/pii_detector.py`:
1. Implement detection for:
   - Emails (RFC 5322 regex)
   - Indian Mobile Numbers (+91, 91, or 10 digits starting with 6-9)
   - Credit Cards (15-16 digits)
   - Indian Aadhaar Numbers (12 digits: XXXX XXXX XXXX or 12 continuous digits; exclude repeated digits like 0000 0000 0000)
   - Indian PAN Cards (5 uppercase letters + 4 digits + 1 uppercase letter, e.g., ABCDE1234F)
2. Add support for Hinglish context cues ("mera aadhaar", "apna phone", "card number") that boost confidence score.
3. Return a list of `DetectedSpan` dataclass objects with: category, start, end, matched_text, confidence, and replacement token (e.g. <AADHAAR_REDACTED>).
4. Include clean pytest unit tests in `tests/unit/test_detection.py`.
```

PROMPT 1.2: Secret Scanner & Prompt Injection Detector — **Target: Cursor / ChatGPT-4o**

```
You are a security engineer implementing credential and jailbreak defense for Sentinel AI.
Create two Python modules:
1. `backend/detection/secret_scanner.py`:
   - Detect AWS keys (AKIA[0-9A-Z]{16})
   - Detect OpenAI keys (sk-[a-zA-Z0-9-]{20,})
   - Detect GitHub tokens (ghp_[a-zA-Z0-9]{36})
   - Detect Private Keys (-----BEGIN RSA/OPENSSH PRIVATE KEY-----)
   - Detect plain passwords (password="...", db_pass="...")
2. `backend/detection/injection_detector.py`:
   - Detect direct prompt injection patterns: "ignore previous instructions", "you are now DAN", "reveal system prompt", "bypass safety filters".
3. Write test cases verifying that normal coding questions are NOT falsely flagged.
```

PROMPT 1.3: Policy Engine & Block-Before-Forward Orchestrator **Target: Claude 3.5 Sonnet / Antigravity**

```
Implement `backend/policy/engine.py` and `backend/gateway/orchestrator.py` for Sentinel AI.
Requirements:
1. Policy Precedence:
   - If any SECRET or PROMPT_INJECTION is detected -> Decision is BLOCK. Sanitized prompt is None.
   - If PII detected -> Decision is MASK. Transform text by replacing spans in REVERSE index order (right-to-left)
to avoid offset shifting.
   - If clean -> Decision is ALLOW.
2. Gateway Orchestrator:
   - Call detectors -> evaluate policy.
   - ENFORCE BLOCK-BEFORE-FORWARD (FR-005): If BLOCK, stop immediately. DO NOT invoke provider adapter. Log
metadata audit event with provider_called=False.
   - If ALLOW/MASK, invoke provider adapter and output sanitizer.
3. Write `tests/security/test_block_before_forward.py`: Assert that when a prompt contains an AWS key,
provider_adapter.invocation_count remains 0!
```

4 Member 2 Deep Dive: Integration & Retrieval Engineer

Primary Objective: Build the server-side LLM provider integration, enforce document permissions before RAG retrieval, and inspect model outputs for leaks.

Owned Files & Structure:

- `backend/providers/base.py` & `backend/providers/adapter.py`: Upstream provider caller (OpenAI, Groq, Anthropic) with local mock fallback, timeouts, and invocation counters.
- `backend/access_control/acl.py`: Pre-retrieval authorization checking user role and allowed document scopes.
- `backend/retrieval/doc_store.py`: Document catalog (`public`, `marketing_general`, `hr_confidential`, `financial_records`); wraps documents in untrusted data delimiters.
- `backend/output_filter/sanitizer.py`: Inspects provider output before returning to user; redacts PII or blocks credential regurgitation.
- `backend/api/routes.py`: REST API endpoints: `POST /v1/chat/completions`, `GET /health/live`, `GET /health/ready`.
- `tests/security/test_acl.py`: Tests verifying pre-retrieval scope enforcement.

Member 2 AI Prompts (Copy-Paste Ready):

PROMPT 2.1: Server-Side LLM Provider Adapter with Mock Fallback — **Target: Cursor / Claude 3.5 Sonnet**

```
You are writing the LLM integration layer for Sentinel AI at `backend/providers/adapter.py`.
Requirements:
1. Class `LLMProviderAdapter(BaseLLMProvider)`:
   - Support OpenAI and Groq APIs via HTTPX using server-side keys (LLM_API_KEY).
   - Server-side key must never be exposed in client responses or logs.
   - Enforce configurable timeout (REQUEST_TIMEOUT_SECONDS, default 15s).
2. Offline Mock Fallback:
   - If LLM_PROVIDER=mock or API key is unset, generate realistic contextual answers locally without network calls.
3. Security Verification Attribute:
   - Track `self.invocation_count` so security test suites can verify whether the model was invoked.
```

PROMPT 2.2: Pre-Retrieval Document ACL & RAG Isolation — **Target: Cursor / ChatGPT-4o**

```
Implement `backend/access_control/acl.py` and `backend/retrieval/doc_store.py` for Sentinel AI.
Requirements:
1. FR-008 Compliance: Document authorization MUST be checked in backend code BEFORE retrieval. The LLM must NEVER
decide document permissions.
2. `AccessControlService`:
   - Checks if user's `allowed_doc_scopes` contains the requested scope.
3. `DocumentStore`:
   - Store mock documents classified as: "public", "marketing_general", "hr_confidential", "financial_records".
   - `retrieve(user, requested_scopes)`: Filters documents against authorized scopes and omits forbidden docs
BEFORE context construction.
   - Wrap retrieved text in untrusted context boundary tags to guard against indirect prompt injection.
4. Write `tests/security/test_acl.py`: Verify that a standard user with ["public"] scope cannot retrieve HR
confidential files.
```

PROMPT 2.3: Post-Generation Output Sanitizer & FastAPI Endpoints — **Target: Claude 3.5 Sonnet / Antigravity**

```
Write `backend/output_filter/sanitizer.py` and `backend/api/routes.py` for Sentinel AI.
Requirements:
1. Output Sanitizer:
   - Scan model responses. If the model outputs secrets/passwords, block the entire response and return a security
warning.
   - If the model outputs PII (emails, phone, Aadhaar/PAN), redact them before returning.
   - Enforce fail-closed safety: on unexpected errors, never return uninspected raw text.
2. FastAPI Routes:
   - `POST /v1/chat/completions`: Accepts GatewayRequest, calls orchestrator, returns GatewayResponse.
   - `GET /health/live`: 200 OK without disclosing secrets.
   - `GET /health/ready`: Checks DB and policy engine readiness.
```

5 Member 3 Deep Dive: Full-Stack & QA Engineer

Primary Objective: Guarantee zero-leakage audit logging, build the 120+ case evaluation benchmark, construct the interactive Web UI, and handle Dockerization.

Owned Files & Structure:

- `backend/audit/models.py` & `backend/audit/logger.py`: Privacy-preserving metadata audit logger (SQLite); strictly guarantees zero raw prompt or response logging.
- `tests/evaluation_corpus/dataset.json`: 120+ curated security, PII, secret, and Hinglish test cases.
- `backend/evaluation/benchmark.py`: Benchmark runner computing Precision, Recall, False-Positive Rate, Prevention Rate, Latency (p_{50}, p_{95}), and verifying zero raw leaks in the DB.
- `frontend/index.html`: Modern Single Page Application with interactive chat, real-time security decision badge, "What the LLM received" diff view, role switcher, and live audit log drawer.
- `deploy/Dockerfile` & `compose.yaml`: Docker setup for one-command execution.

Member 3 AI Prompts (Copy-Paste Ready):

PROMPT 3.1: Zero-Leakage Metadata Audit Logger — **Target: Cursor / Claude 3.5 Sonnet**

```
Implement `backend/audit/logger.py` and `backend/audit/models.py` for Sentinel AI.
CRITICAL PRIVACY RULE: Default logs must contain ZERO raw prompt text, ZERO raw LLM output, and ZERO credentials.
Requirements:
1. Define `AuditEvent` schema: event_id, timestamp, tenant_id, pseudonymous_user_id, action_taken
(ALLOW/MASK/BLOCK), rule_categories, policy_version, provider_called, gateway_latency_ms, total_latency_ms.
2. Persist metadata to SQLite table `audit_events` and keep recent 50 events in memory for fast UI polling.
3. Implement `get_recent_events(limit)` for admin dashboard queries.
4. Add verification logic that checks the DB to ensure no raw secrets or PII strings ever exist in any column.
```

PROMPT 3.2: 120+ Case Evaluation Dataset & Benchmark Runner — **Target: Cursor / ChatGPT-4o**

```
Create the automated security evaluation suite for Sentinel AI:
1. `tests/evaluation_corpus/dataset.json`:
   - 120+ test cases across: benign queries, PII (email, phone, credit card), Indian Aadhaar & PAN cards, secrets
(AWS, OpenAI, GitHub, passwords), prompt injections (DAN, jailbreaks), and Hinglish prompts.
2. `backend/evaluation/benchmark.py`:
   - Execute all dataset cases against `gateway_orchestrator.process_chat()`.
   - Calculate and display:
     * Precision (TP / (TP + FP))
     * Recall (TP / (TP + FN))
     * False-Positive Rate (FPR)
     * Secret Prevention Rate (Must be 100%)
     * Gateway Latency Overhead (p50 and p95 in ms)
     * Raw Content Check in DB (Must be 0)
   - Save summary to `eval_results_latest.json`.
```

PROMPT 3.3: Interactive Web Chat & Security Inspector Frontend — **Target: Claude 3.5 Sonnet / Antigravity**

```
Build a modern, dark-mode Single Page Application at `frontend/index.html` using Tailwind CSS and Vanilla JS.
Features:
1. Left Panel (Chat Stream):
   - Instant test scenario buttons: "Benign Code", "Aadhaar + Phone (MASK)", "PAN + Card (MASK)", "AWS Key (BLOCK)", "Prompt Injection", "Hinglish PII".
   - Clean chat message history with user and AI bubbles.
2. Right Panel (Gateway Security Inspector):
   - Decision Badge: ALLOW (green), MASK (amber), BLOCK (red).
   - "Provider Called?" Indicator: Green pulse (Yes) vs Red shield (NO - Blocked before forward).
   - Gateway Latency overhead display (ms).
   - Triggered threat/PII category chips.
   - "What the LLM Received" diff view showing masked text or block explanation.
3. Top Navigation:
   - Role switcher ("End User" vs "Security Reviewer/Admin").
   - Doc scope selector ("None", "Public", "HR Confidential", "Financial").
   - "Audit Logs" button opening a modal with live structured metadata table.
4. Dual-Mode Fallback: If port 8000 is running, fetch from API; otherwise run client-side emulation so the team can demo immediately!
```

6 5-Week Sprint Roadmap & Integration Checkpoints

Week	Milestone Focus	Member 1 Focus	Member 2 Focus	Member 3 Focus
Week 1	**Vertical Slice & Skeleton**	FastAPI app setup, JWT models, request validation schemas.	Provider adapter with mock fallback and request timeout.	Repository setup, SQLite schema, zero-raw-log design.
Week 2	**Detection & ACL Engine**	PII (Email/Phone/Aadhaar/PAN) and secret scanners.	Pre-retrieval ACL logic and document store catalog.	Curate first 50 cases in evaluation dataset.
Week 3	**Enforcement & Filtering**	Policy engine, right-to-left masking, block-before-forward.	Output sanitizer and post-generation leak inspection.	Evaluation metrics runner (Precision/Recall calculations).
Week 4	**Hinglish & Benchmark**	Hinglish detectors and injection override heuristics.	REST endpoints finalized and error boundaries secured.	Expand to 120+ cases; run benchmark; verify 0 DB leaks.
Week 5	**UI, Docker & Live Demo**	Adversarial test review and security edge cases.	Connect live OpenAI/Groq API key for production demo.	Polish Web UI, build Docker image, record live demo.

7 Git Branching Strategy & Collaboration Workflow

To avoid conflicts while all 3 members work concurrently:

- `main`: Production-ready baseline. All tests must pass before merging.
- `feature/member1-gateway-detection`: Owned by Member 1 (Auth, Detection, Policy, Orchestrator).
- `feature/member2-provider-acl`: Owned by Member 2 (LLM Adapter, ACL, DocStore, Output Filter).
- `feature/member3-audit-ui-eval`: Owned by Member 3 (Audit Logger, Benchmark Suite, Web UI, Docker).

> **Standard Pull Request (PR) Acceptance Checklist:**
>
> 1. `pytest tests/ -v` passes with 0 failures.
> 2. Block-Before-Forward test confirms provider call count is 0 on blocked prompts.
> 3. Pre-retrieval ACL test confirms unauthorized documents are omitted.
> 4. Audit DB verification confirms 0 raw sensitive prompts exist in storage.